DL26, Assignment 2

Richard Whittingham Plummer (rplummer, 2037014)

April 26, 2026

Task 2a)

Upon inspecting the dataset in the shared repository, it becomes apparent how the dataset instance is generated. The dataset generation process begins with 10 distinct 12 coordinate templates that represent the shape of each digit. These templates are first standardized by subtracting their mean to center the signal around zero and dividing by their standard deviation for normalization purposes. The signal is then shifted such that the first value is exactly zero and is further scaled by a factor of 1/6. To create individual dataset instances, these normalized templates then undergo a transformation (make dataset function) where they undergo a random translation, scaling, and the addition of iid noise. Throughout this process, each generated signal is paired with its corresponding digit label from the original template to ensure the identity of the digit is preserved through the transformations. Finally, the signals are padded with zeros to reach a total sequence length of 40, resulting in a diverse set of samples that share a consistent structure. Because the dataset consists of 5000 rows, each digit undergoes the transformation 500 times.

Task 2c)

To compute the number of parameters, I use the formula for a 1D-convolutional layer:

$$P_{conv} = ((C_{in} * K) + 1) * C_{out} \tag{1}$$

$$P_{linear} = (L_{in} + 1) * L_{out}. \tag{2}$$

(i) Parameters in each layer:

- **conv1**: $((1 \times 3) + 1) \times 5 = 20$
- **conv2**: $((5 \times 3) + 1) \times 5 = 80$
- **conv3**: $((5 \times 3) + 1) \times 5 = 80$
- **conv4**: $((5 \times 3) + 1) \times 5 = 80$

I use the following formula to calculate the convolutional output size:

$$L_{out} = \lfloor \frac{L_{in} + 2P - K}{S} + 1 \rfloor \tag{3}$$

- After conv1: $\lfloor \frac{40+2-3}{2} + 1 \rfloor = 20$
- After conv2: $\lfloor \frac{20+2-3}{2} + 1 \rfloor = 10$
- After pool1 ($S = 2$): $10/2 = 5$
- After conv3: $\lfloor \frac{5+2-3}{2} + 1 \rfloor = 3$
- After conv4: $\lfloor \frac{3+2-3}{2} + 1 \rfloor = 2$
- After pool2 ($S = 2$): $2/2 = 1$

The input to the linear layer is $linear_in = Channels \times Length = 5 \times 1 = 5$.

- **linear**: $(5 + 1) \times 10 = 60$

(ii) Overall parameters: Total $= 20 + 80 + 80 + 80 + 60 =$ **320**

Task 2d): i), ii), and iii)

i) According to the provided link, map-style based datasets are preferred when working with the MNIST-1D dataset. The map-style dataset is more suitable because it implements 'getitem' and 'len', allowing the DataLoader to access any sample by its index for efficient retrievals. Furthmore, because of the simple structure of the dataset, one can easily access the label via 'data[index]'. Unlike Iterable-style datasets, which are designed for massive, continuous data streams where the total size is unknown, MNIST-1D is a small, fixed-size dataset that fits easily into memory.

ii) After referring to the provided link on the tensor dataset source code, I determine that the tensor dataset is a map-style dataset. This is due to the fact that it has the two functions, len and getitem, both of which are markers of map-style datasets.

iii) The primary reason to use CrossEntropyLoss over NLLLoss is that it combines LogSoftmax and NLLLoss into a single numerically stable loss function, which prevents mathematical problems like numerical underflows or overflows by utilizing the "Log-Sum-Exp" trick. NLLLoss expects log-probabilities as an input which would require the architecture to have a log softmax at the end of the foward pass. Since the simple CNN is, as the name suggests, a simple architecture, using NLLLoss would only make the architecture more complicated than it would need to be.

Task 2e)

In order to calculate the amount of parameters of the FNN, I multiply the number of weights with the amount of input nodes and add the amount of bias values, such that:

$$|P_{MLP}| = \sum_{i=1}^{3} \left(p_{in}^{(i)} \cdot p_{out}^{(i)} + p_b^{(i)} \right) \tag{4}$$

Using the above formula and looking at architecture of the class, I calculate $(40 * 256 + 256) + (256 * 256 + 256) + (256 * 10 + 10) = 78858$.

Given my previous knowledge on CNNs from the lectures, I expect the CNN to significantly outperform the FNN given that CNN are architectures built around visual object classification given the used channel windows.

Task 2f)

After training the simple MLP and analzying the results, as predicted, the CNN had a much higher test accuracy of 90.6 versus the MLP's test accuracy of 64.7. This is expected because CNNs learn local structure in the input through the previously mentioned filters, making them well-suited for 1D signals. The MLP, despite having more parameters (78,858), treats the input as a flat vector.

Task 2h)

After shuffling the dataset, the original forty coordinates are permutated, now forming a random sequence. I believe that the CNN's inductive bias, as covered in the lecture, which assumes that local pixels are related will reduce the performance of the model. Since the local points no longer have an inherent relationship, the CNN will most likely learn "incorrect" features. Since the FNN doesnt have this inductive bias, its performance will likely stagnate compared to the CNN.

Task 2i)

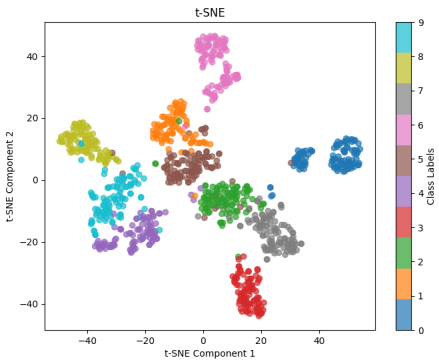
As expected, the performance of the CNN drastically declined. It previously achieved a test accuracy of 90.6 and now has dropped to 55.3 percent. On the other hand, the FNN's performance has also gotten worse. Instead of the previous 64.7 percent, its test accuracy has dropped to 62.1 percent. I was correct with my conjecture about the CNN's performance, yet it's surprising that the accuracy of the model is still at 55.3 percent given that the order of the MNIST-1D rows are random. The surprising performance decline of the FNN is also likely to be attributed to the random permutation of the values.

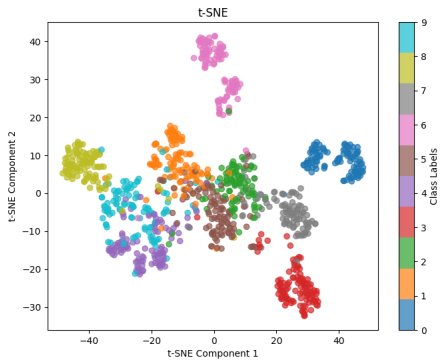
Task 3c)

(i) I expect the raw input data to show little to no clear cluster separation, with classes largely mixed together. The raw 40-dimensional signals have no learned representation yet. (ii) I expect to see clearer cluster separation compared to the raw input data. Since the CNN was trained on this data, it should have learned to map similiar class inputs close together in the embedding space. (iii) I expect the test embeddings to look similar to the training embeddings, but with slightly less tight clusters and potentially more overlap at class boundaries. The degree of similarity between the training and test embeddings would indicate how well the model generalizes.

Task 3d)

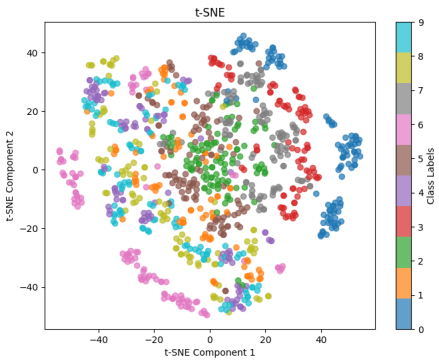
The t-SNE plots of the training and test embeddings reveal several findings that largely confirm my prior conjectures. Below are said graphs with the left figure representing the training embeddings while the right depicts the test embeddings. For the experiment, I do not tweak the perplexity as this value likely would not change the cluster distances. As expected, the training embeddings show clearly separated clusters with greater distances between them, which is consistent with the CNN having been optimized directly on this data and learning to map same class examples far apart in embedding space. While the training embedding figure has individual points touch other clusters, the overall classification yields promising results. The test embeddings, while showing some clearly separable clusters, also show cases where clusters are partially merged, indicating that the model does not perfectly generalize to unseen data. This is consistent with the above conjecture that test embeddings would show slightly less tight clusters with more overlap at class boundaries compared to the training embeddings. The biggest overlap is noticable between the classes 9 and 4. The model seems to have trouble distinguishing the two as the two clusters virtually depict the same cluster. Furthermore, both plots show better cluster structure than the raw input data t-SNE plot, emphasizing that the CNN successfully learns to transform an initially unstructured input space into a more separable embedding space. It's important to note that the test accuracy of of the simple CNN class is 90.6 percent which can account for the 9.4 percent overlapping clusters.

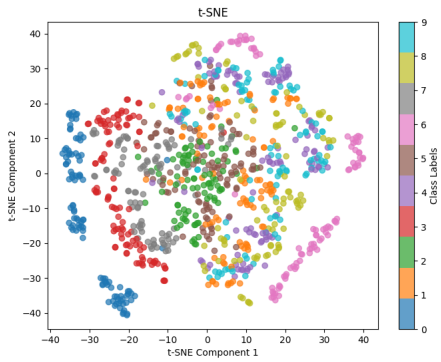




Task 3e)

Given that there are two convolution blocks before the linear prediction head, then there is only one more embedding layer to analyze. The resulting figures represent the corresponding training and test embedding T-SNE graphs of the first layer with the left representing train and the right test. Compared to the second layer embeddings, the first layer shows noticeably less separated clusters, with more overlap and mixing between classes, which is expected since the first conv-block has only partially processed the input and has not yet learned the full representation. However, the clusters are still significantly more structured than the raw input data t-SNE plot, indicating that even the first conv-block extracts meaningful features that begin to separate the classes. The test embeddings of the first layer follow a similar pattern to the training embeddings, with comparable cluster structure but slightly more overlap, consistent with earlier findings. Interesting to note is that in both figures, the classes 6, 0, 3 seem to have established clusters already, perhaps suggesting that these classes may be easier to learn. This progression from the first to the second layer highlights that the deeper layers are necessary to learn the final representations, with class separation improving as the signal passes through more convolutional blocks.





Task 3f)

Task 4)

Modifying the hyperparameters shows how much impact they have toward the amount of parameters and test accuracies. Below is a table summarizing the experimental results. Regarding the learning rate and batch size (4a), the baseline configuration already uses well-suited values, as both alternatives perform significantly worse. Using a lower learning rate of 0.001 with a smaller batch size leads to underfitting within 50 epochs which results in an accuracy of only 71.1 percent, while the higher learning rate of 0.1 causes unstable training that fails to converge correctly, achieving only a 38.5 percent accuracy. For kernel size (4b), increasing to kernel size 5 slightly improves accuracy to 92.6 percent, while reducing to kernel size 1 effectively removes any notion of locality and substantially reduces the accuracy to 16.5 percent, confirming that local feature extraction is critical for this task. For the number of channels (4c), reducing to 10 channels reduces the performance at 60.3 percent due to insufficient model capacity, while increasing to 50 channels achieves 91.6 percent which is close to the baseline performance. Despite having nearly four times as many parameters, increasing the amount of channels beyond 25 seems to suggest diminishing returns. The stride experiments (4d) produce the best results. Reducing stride to 1 achieves the highest test accuracy of 97.1 percent, while increasing stride to 3 hurts performance at 78.4 percent. Overall, the most suitable design choice appears to be stride=1 with default values for all other hyperparameters, as it achieves the best test accuracy of 97.1 percent at a reasonable parameter count of 8,310, demonstrating that preserving spatial resolution may be the most impactful choice for this task.

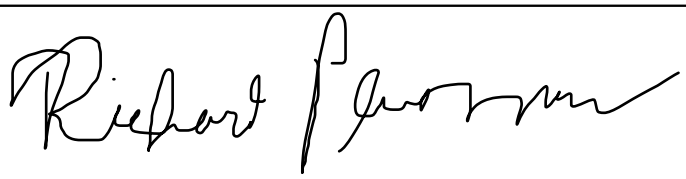
Subtask	**Changed Parameters**	**Params**	**Test Acc**
Baseline	ch=25, ks=3, st=2, lr=0.01, bs=64, ep=100	6,060	90.6
4a) Option 1	lr=0.001, bs=32, ep=50	6,060	71.1
4a) Option 2	lr=0.1, bs=128, ep=150	6,060	38.5
4b) Option 1	ks=5, pad=2	9,860	92.6
4b) Option 2	ks=1, pad=1	2,260	16.5
4c) Option 1	ch=10, linear_in=10	1,080	60.3
4c) Option 2	ch=50, linear_in=50	23,360	91.6
4d) Option 1	st=1, linear_in=250	8,310	97.1
4d) Option 2	st=3, pad=2	6,060	78.4

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Task	Solved with AI?	Tool	Purpose	Useful?
1a				
1b				
2a				
2b				
2c	yes	Gemini 3.1	LaTex formatting + I couldn't find formulas for calculating parameters in lecture slides or exercise	yes
2d				
2e	no	Gemini 3.1	LaTex formatting for equations	yes
2f				
2g				
2h				
2i				
3a				
3b				
3c				
3d	no	Gemini 3.1	LaTex formatting for figures	yes
3e	no	Gemini 3.1	LaTex formatting for figures	yes
3f				
4	no	Gemini 3.1	LaTex formatting + Writing codeblock for determining "linear in" without needing to do it by hand each time	yes



Unterschrift
Mannheim, den 26.04.2025